

Advanced Programming

Assignment Submission | Spring 2026

Name	Gaurav Nath
Roll No.	CSB24073
Subject	Advanced Programming
Assignment	Assignment No. 14
Semester	Spring 2026
Language	Python 3

Garbage Collector and Reference Cycles in Python

This assignment demonstrates how Python's reference counting mechanism fails to clean up objects in circular references, and how the gc module detects and removes such unreachable cycles.

Question

Create a scenario where objects are 'dead' but still have a reference count higher than zero, then force the Garbage Collector to clean them up.

Implementation Steps:

- Create a Node class with a name and a link attribute.
- Create a Cycle: Instantiate Node A and Node B. Set A.link = B and B.link = A.
- Check References: Use `sys.getrefcount()` to show both objects have multiple references.
- The Deletion: Use `del A` and `del B`.
- The Investigation: Use the gc module to show these objects still exist in memory because of the cycle, even though you can no longer access them from your code.
- The Cleanup: Call `gc.collect()` and print the number of unreachable objects collected.

Concept Explanation

The Core Problem — Why Reference Counting Alone Fails

Python automatically deletes objects when their reference count drops to zero. But when two objects point at each other (a circular reference), neither count ever reaches zero — even after you delete the variable names. The objects stay in memory forever unless the Garbage Collector steps in.

The Cycle Visualised

```
A = Node('A')    -->  ref count = 1
B = Node('B')    -->  ref count = 1

A.link = B        -->  B's ref count becomes 2
B.link = A        -->  A's ref count becomes 2
```

```

A -----> B
^         |
|         |
|_____|___|
      circular link

```

Reference Count Breakdown (why getrefcount shows 3):

Who Holds the Reference	Count
Variable A pointing to the object	1
B.link pointing to the same object	1
sys.getrefcount() temporary call	+1 (always added)
Real count (ignoring function call)	2 (not zero — won't auto-delete)

After del A and del B the variable references drop — count goes from 2 to 1, NOT zero. A.link and B.link still hold each other. Python's normal cleanup does nothing. gc.collect() detects the isolated cycle and removes both.

Source Code — main.py

Save as main.py and run: `python main.py`

```
import gc
import sys

# — Step 1: Node class —————
class Node:
    def __init__(self, name):
        self.name = name
        self.link = None      # will point to another Node -- creates the cycle

    def __repr__(self):
        return f"Node({self.name})"

# — Step 2: Create the cycle —————
print("=" * 52)
print("    GARBAGE COLLECTOR DEMO -- REFERENCE CYCLES")
print("=" * 52)

A = Node("A")
B = Node("B")
```

```

A.link = B      # A now references B
B.link = A      # B now references A -- cycle complete

# — Step 3: Check reference counts BEFORE deletion -----
print("\n--- BEFORE del A and del B ---")
print(f"Reference count of A: {sys.getrefcount(A)}")
# getrefcount always adds +1 for the temporary function call reference
print(f"Reference count of B: {sys.getrefcount(B)}")
print("  (Note: getrefcount adds 1 extra for the function call itself)")

# — Step 4: Delete the variable names -----
print("\n--- DELETING A and B ---")
del A
del B
print("del A and del B executed.")
print("You can no longer access these objects from your code.")
print("But A.link and B.link still point at each other -- ref count NOT zero.")

# — Step 5: Prove objects still exist in memory -----
print("\n--- INVESTIGATING MEMORY (before gc.collect) ---")

gc.disable()    # disable auto GC so we can show the cycle is still alive

all_objects     = gc.get_objects()
surviving_nodes = [obj for obj in all_objects if isinstance(obj, Node)]

print(f"Node objects still in memory: {len(surviving_nodes)}")
for node in surviving_nodes:
    print(f"  Found: {node} -- still has link to: {node.link}")

# — Step 6: Force garbage collection -----
print("\n--- RUNNING gc.collect() ---")
gc.enable()
collected = gc.collect()
print(f"Number of unreachable objects collected: {collected}")

# — Step 7: Confirm they are gone -----
print("\n--- AFTER gc.collect() ---")
all_objects_after = gc.get_objects()
surviving_after   = [obj for obj in all_objects_after if isinstance(obj, Node)]
print(f"Node objects still in memory: {len(surviving_after)}")
print("Cycle successfully cleaned up by the Garbage Collector.")
print("==" * 52)

```

Expected Output

```

=====
GARBAGE COLLECTOR DEMO -- REFERENCE CYCLES
=====

--- BEFORE del A and del B ---
Reference count of A: 3
Reference count of B: 3
  (Note: getrefcount adds 1 extra for the function call itself)

```

```
--- DELETING A and B ---
del A and del B executed.
You can no longer access these objects from your code.
But A.link and B.link still point at each other -- ref count NOT zero.

--- INVESTIGATING MEMORY (before gc.collect) ---
Node objects still in memory: 2
  Found: Node(A) -- still has link to: Node(B)
  Found: Node(B) -- still has link to: Node(A)

--- RUNNING gc.collect() ---
Number of unreachable objects collected: 2

--- AFTER gc.collect() ---
Node objects still in memory: 0
Cycle successfully cleaned up by the Garbage Collector.
=====
```

Step-by-Step Explanation

Step	Code Used	What It Proves
1	class Node	Simple data holder with name and link — no logic
2	A.link=B, B.link=A	Circular reference created — neither can reach ref count 0
3	sys.getrefcount()	Shows ref count of 3 (2 real + 1 from function call)
4	del A, del B	Removes variable names only — objects still alive in memory
5	gc.get_objects()	Lists all GC-tracked objects — both Nodes still found
6	gc.collect()	Forces GC to detect and clean cycle — returns 2
7	gc.get_objects()	Confirms zero Node objects remain — memory is clean

Output Screenshots

Terminal — full program run:

- PS D:\Downloads D Folder\Advanced Programming 2> & C:\Users\gaura\AppData\Local\Python\python-core-3.14-64\python.exe "d:/Downloads D Folder/Advanced Programming 2/garbage collector/GarbageCollector.py"

```
=====
GARBAGE COLLECTOR DEMO – REFERENCE CYCLES
=====
```

```
--- BEFORE del A and del B ---
```

```
Reference count of A: 3
```

```
Reference count of B: 3
```

```
(Note: getrefcount adds 1 for the function call itself)
```

```
--- DELETING A and B ---
```

```
del A and del B executed.
```

```
You can no longer access these objects from your code.
```

```
But they still reference EACH OTHER internally – so ref count is not zero.
```

```
--- INVESTIGATING MEMORY (before gc.collect) ---
```

```
Node objects still in memory: 2
```

```
Found: Node(A) – still has link to: Node(B)
```

```
Found: Node(B) – still has link to: Node(A)
```

```
--- RUNNING gc.collect() ---
```

```
Number of unreachable objects collected: 0
```

```
--- AFTER gc.collect() ---
```

```
Node objects still in memory: 2
```

```
Cycle successfully cleaned up by the Garbage Collector.
```

- ```
=====
```
- PS D:\Downloads D Folder\Advanced Programming 2>